

Recursive BOM with Routing in Manufacturing | by Tim Tsai | Jul, 2025 | Medium

By Tim Tsai

Recursive BOM with Routing in Manufacturing

Recursive BOM with Routing in Manufacturing

The Manufacturing is Coming Back, but... What are the Real Bottlenecks for Manufacturing

Enterprise Resource Planning (ERP) systems are at the heart of modern manufacturing — yet one of their most critical features often becomes their greatest bottleneck: the Bill of Materials (BOM). In manufacturing, a BOM is a hierarchical list of components and subcomponents required to build a finished product. It's essentially a blueprint of how things are put together — from nuts and bolts, all the way to final assemblies. Equally important is the routing: the sequence of operations required to transform raw materials into a finished good. Each routing step — like cutting, drilling, welding, or polishing — adds time, cost, and complexity to production. Take the metal hardware manufacturing as an example... Ever wonder what causes the most product complaints at Louis Vuitton repair centers? Surprisingly, it's the snap hook of the boutique bags — a small part that breaks more often than you'd think. Routings can be particularly tricky: Welding operations may have lower yield rates due to precision challenges. Polishing might require repeated passes for quality assurance. Heat treatment can cause dimensional changes, requiring rework or scrap. These realities aren't always captured cleanly in ERP systems, especially in platforms like SAP Business One (SAP B1) which offer basic BOM support but lack deep recursive analytics or built-in routing cost calculations. That's where integration comes in: by combining BOM hierarchy with routing details, we can compute:

- Total raw material quantities needed (including nested levels)
- Labor and machine time per component
- Process-specific cost per finished unit
- Yield-adjusted material planning

Such analysis solves real operational pain points:

- Prevents over-ordering of raw materials
- Identifies high-cost operations in the process chain
- Improves quotation accuracy
- Enables predictive scheduling and resource allocation

During my time at a global metal manufacturing subsidiary, I tackled this exact challenge — building a recursive BOM + routing analysis engine from the ground up. I believe the approach would help those companies with limited resources and budget.

The First Challenge: BOM Expansion in SAP

Unlike flat BOMs used in simple manufacturing, SAP stores Bills of Materials (BOMs) as hierarchical trees. Each finished good (FG) may consist of subassemblies (SFGs), which in turn have multiple raw materials, each with its own quantities. Consider this example: To compute the total required material quantity, you need a recursive traversal of this tree: This is not natively supported in SAP (B1) or most ETL tools out of the box.

Recursive Structures and Mathematical Thinking

Before jumping into tools, I first asked: What kind of problem is this, structurally? At its core, this recursive BOM expansion is a tree traversal problem — every finished product (FG) is the root of a tree, and each semi-finished product (SFG) or material is a node, linked by “parent-child” relationships with quantity multipliers.

Conceptual Model: Each node in the tree represents a component, and the total quantity needed at the leaf node (raw material) is:

Repeated through recursion, this is equivalent to:

This structure is mathematically similar to:

- Graph Theory: Specifically, a directed acyclic graph (DAG) where each edge has a weight (the Quantity)
- Tree-based Dynamic Programming: Where solutions are built bottom-up or top-down
- Matrix Chain Multiplication (in reverse): Where the cumulative impact is multiplicative across ordered nodes

The trickiest part was not the SQL syntax, but:

- Ensuring that shared components (used by multiple FGs) didn't double-count
- Preventing cyclic dependencies (rare but possible in poorly maintained BOMs)
- Handling deep trees (up to 11 layers!) within time limits in ETL runs

This was not just a data problem. It was a graph problem under business constraints.

Sample SQL Recursive BOM Expansion with Routing Costs

My Solution

Using a combination of SQL recursive queries and Pentaho ETL workflows, I created a system that:

- Pulled each top-level finished good (OITM table, ItemCode starts with

'B') Wrote it into a staging table (MySQL_FG_List) with aCheckFlag = 'N' Used a looped process to:

1. Query each BOM level from SAP B12. Multiply material quantities recursively
3. Aggregate results into aTotalQTYcolumn
4. Mark entries asCheckFlag = 'Y' to prevent reprocessing

Triggered email alerts on query failures for data integrity

The workflow chart for ETL tasks: Output: Multi-level material breakdowns rendered in reporting with drill-down summaries.

Why Pentaho Was Limiting — And How I'd Rebuild It + Routing Synergy

While Pentaho served well as an introductory ETL tool, it fell short when scaling to complex multi-tier integrations between BOM and Routing structures. In a robust manufacturing intelligence system, understanding what materials are needed (BOM) is only half the equation. You also need to know how they are processed (Routing)— each routing step adds cost, labor, and time.

The Integrated BOM-Routing Model In SAP B1 and similar ERP systems

two key tables underpin production planning: BOM Table (OITT + ITT1): defines which components go into a parent item, including quantities Routing Table (OITT joined with OWOR + WOR1): defines how the item is produced, with steps including work centers, standard times, and cost centers

To compute total cost and time, I constructed an integrated model where each semi-finished item (SFG) would:

- Be expanded recursively using BOM logic
- For each node (item), retrieve the routing sequence (StepID, OperationCode)
- From each operation, pull:
 - Labor Hours × Hourly Rate = Labor Cost
 - Machine Hours × Rate = Machine Cost
 - Material Scrap Rate and Yield Efficiency
- Aggregate recursively back to the top-level product

Example Logic Flow: From this flow, we compute: M001—1 appears: $2 \times 3 \times 2 = 12$ units M001 operation time: $3 \times 0.2 = 0.6$ hrs S001 labor: $0.5 + 1.0 = 1.5$ hrs $\rightarrow 2 \times 1.5 = 3.0$ hrs for B001

Costing per routing step is multiplied at each layer and summed upwards

How I'd Rebuild It Today (Modern Stack)

If I were to rebuild this system today, I would:

- Use MySQL recursive CTE to expand the BOM hierarchy
- Use a normalized Routing table with fields: ItemCode, StepNo, Operation, LaborHours, MachineHours, LaborRate, MachineRate
- Build an ETL pipeline in Python:
 - Extract recursive BOM from MySQL
 - Join each component with its routing
 - Compute cost/time/material metrics in pandas
 - Write output to a Data Warehouse table (e.g., ProductionCost_Fact)

How the ETL Logic Works Recursive Expansion with Cost Calculation (Pseudocode):

What it actually does:

Lessons from Refactoring Rebuilding this solution in Python has revealed a powerful truth:

the right tools don't just make systems faster — they make teams stronger.

When we first implemented this using Pentaho, it gave us a quick win with its visual ETL workflow. But over time, we faced growing limitations:

- Complex recursive logic was visually fragmented across multiple jobs
- Query time ballooned as BOM depth increased
- Version control was weak; debugging a pipeline often meant opening 5+ transformation files
- Rapid BOM changes created constant update fatigue

■ Switching to Python changed everything. Recursion was handled naturally via code logic — no more “8-level loops”

The entire ETL was now a single, readable script, trackable in Git

Adding new data columns or altering cost formulas was reduced to minutes

Runtime performance improved dramatically: what took ~8 minutes in Pentaho now finished in under 1 minute

Most importantly, the team could now fully own and improve the codebase — it was no longer a mystery locked inside a GUI.

Looking Forward

Though our Python-based ETL proved highly effective, several advanced options are now within reach:

- Graph Databases (e.g., Neo4j) for modeling recursive BOMs natively with weighted edges
- dbt (Data Build Tool) for SQL-based model versioning and lineage tracking
- Apache Airflow for automated pipeline scheduling and monitoring
- Cloud DWH (Snowflake, BigQuery) for elastic scaling on BOM + routing workloads

There is more than just one solution!